

Lecture 1

Introduction

eecs498-software-design.org



Course Components

- Lecture
- Lab
 - Review, practice exercises, pragmatic tutorials, etc.
 - Project feature checkpoints.
- Projects
- Exams

Icebreaker

- What are you hoping to get out of this course?
- How do you feel about a career in software development?
- What does the future look like for software development?

Learning Objectives

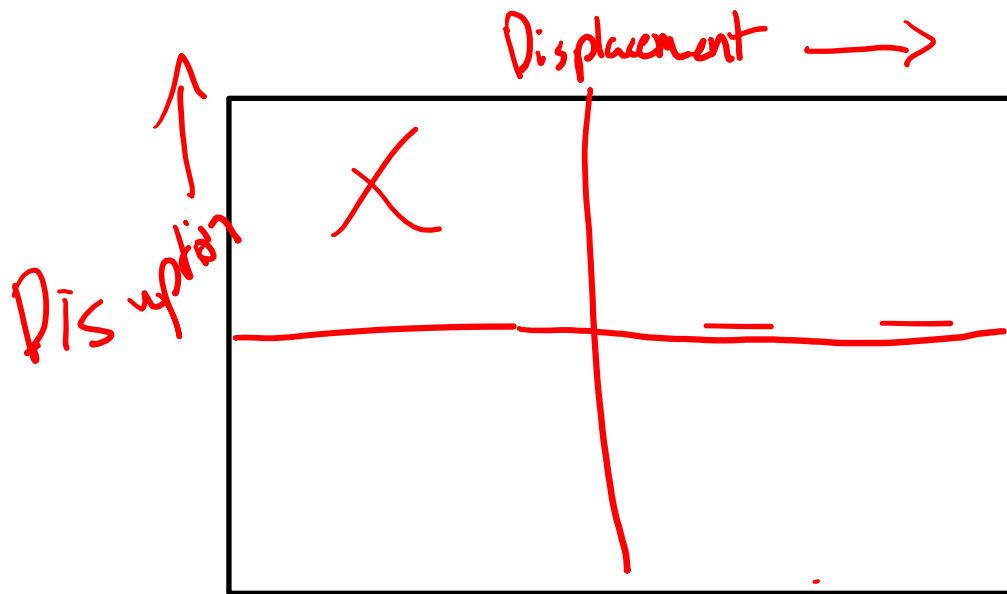
During/after the course, students will be able to...

- ...apply advanced programming techniques to authentic, complex software problems.
- ...develop fundamental reasoning skills for analyzing/writing concurrent software.
- ...appreciate the tradeoffs involved in various approaches to software design.
- ...write more robust, flexible, and maintainable code.
- ...evaluate and explain software quality according to fundamental principles.
- ...identify and implement improvements to poor or overly-complex codebases.
- ...recognize when (and when not) to use established heuristics and design patterns.
- ...engage in continuous improvement and collaborative professionalism in software design.



The Future of Software Development

- AI: Displacement vs. Disruption



Careers in Software Development

- Writing code (and lots of it) has never been easier.
- Perhaps... writing good code has never been more important.
- Job Market (My Guess)
 - Software development will continue to be a viable career path for the foreseeable future, but barriers to entry may be higher and market saturation may take a while to work itself out.



Setting Expectations

- We're building the plane as we fly it...
 - The course will not be as organized as e.g. EECS 280.
 - We may need to make some major adjustment at some point.
 - Our #1 priority is that you have a great experience.
 - Please excuse the delayed syllabus and schedule... coming soon!
- There's no definitive resource on *Software Design Principles*...
 - Let's figure this out together.

Exercise: Software Design Principles

Avoid premature optimization
- choice of DS
- special case

KISS

Keep it
simple stupid

DRY
Don't repeat yourself
Avoid code duplication

Duplication is sometimes
cheaper than the wrong abstraction

code readability
- for others
- separation

avoid globals
(including singletons?)
scalability

SOLID

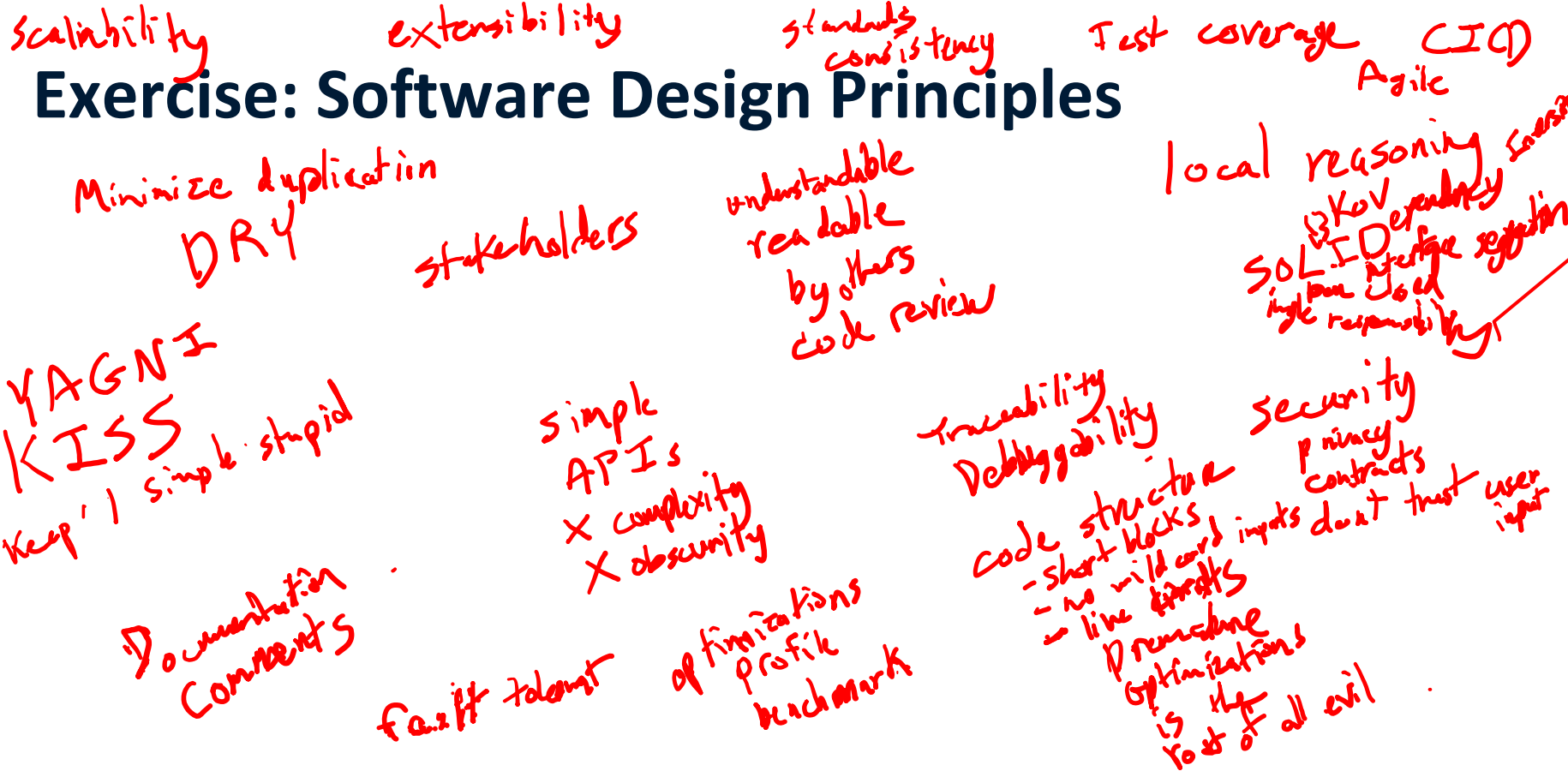
single responsibility
open / closed
Liskov Substitution
Interface segregation
Dependency Inversion

separation of
concerns
information
hiding

TDD

(self) Documentation
Comments





Projects

- **Overall Theme**

Collaborative Puzzle Solving System

- **Project 1 (Individual) – Released 2nd week, Due Fri Jan 23**

Implement a terminal based puzzle game. Choose one:

- Sudoku
- Minesweeper
- Spelling Bee

- **Remaining Projects (Group of 3)**

Combine codebases from P1 to for a puzzle game collection.

Incrementally add features and revise design.



Projects

- Project 2-5 (Group)
- General idea – as the codebase and feature set grow, interesting design challenges emerge.
 - Implementation of cross-cutting features across puzzle games.
 - Graceful extensibility and variation of individual puzzle games.
 - Multiple interfaces, e.g. terminal-based, web-based, admin view.
 - Minimal client-server architecture, database, etc.
 - Play live with other users in collaborative puzzles.
 - Robust error handling.

Project Grading and Feedback

- **Project Feature Checkpoints**

After projects due, demo feature completion in lab.

- **Design Feedback and Grading**

We'll reviewing your code by hand for:

- Design and best practices, especially related to course topics.
- We'll grade your project according to a rubric.
- Detailed feedback and suggestions for improvement.
- Then, you'll have a chance to revise before we grade again.



Typescript Introduction



Typescript

- **Javascript (JS)**

- Originally created in 1995 to add interactivity to web pages.
- Grew to be the dominant language in front-end web dev.
- Expanded to other contexts, e.g. back-end web, scripting, etc.
- **ECMAScript** standardized versions of Javascript, e.g. ES2025

- **Typescript**

- Released in 2012. Goal is to improve dev experience over JS.
- Maintains syntax, semantics, and run-time behavior of JS.
- Adds a type system to detect errors and support dev tools.

Why Typescript for EECS 498 APSD?

- Modern, popular language with significant industry use.
- Acceptable learning curve and mostly familiar syntax.
- Readily supports many paradigms and patterns.
- Other languages worth considering:
 - Java, Kotlin, C#, Rust

Typescript Fundamentals

Demo

Next Steps

- **Typescript Handbook**

<https://www.typescriptlang.org/docs/handbook/intro.html>

- **Lab 1**

Computer Setup

GitHub/Pragmatics

Typescript Practice

- **Project 1, Early Next Week**



Type Systems

- A **type** defines a set of possible values.
 - e.g. **string**: "", "cat", "hello world!"
 - e.g. **boolean**: true, false
 - e.g. **Person**: {"james", 2, 110, 0xE5A4}
 - **Static Typing**:
Variables have a fixed type.
 - **Dynamic Typing**:
A variable may refer to different types.
The type is on the value only.
-
- The job of a type system:
 - What **operations** are available?
 - Which **conversions*** are allowed?
 - **Strong Typing**
Few, if any, implicit conversions.
 - **Weak Typing**
Eh, just do whatever.

*We usually think of this first in terms of **implicit conversions**, sometimes called coercions. Explicit casts may be used to circumvent the usual behavior of the type system (and sometimes its safety guarantees).



Type Systems

Where is C++?

- A **type** defines a set of possible values.
 - e.g. **string**: "", "cat", "hello world!"
 - e.g. **boolean**: true, false
 - e.g. **Person**: {"james", 2, 110, 0xE5A4}
- **Static Typing**:
Variables have a fixed type.
- **Dynamic Typing**:
A variable may refer to different types.
The type is on the value only.
- The job of a type system:
 - What **operations** are available?
 - Which **conversions*** are allowed?
- **Strong Typing**
Few, if any, implicit conversions.
- **Weak Typing**
Eh, just do whatever.

*We usually think of this first in terms of **implicit conversions**, sometimes called coercions. Explicit casts may be used to circumvent the usual behavior of the type system (and sometimes its safety guarantees).



Type Systems

Where is Javascript?

Where is Typescript?

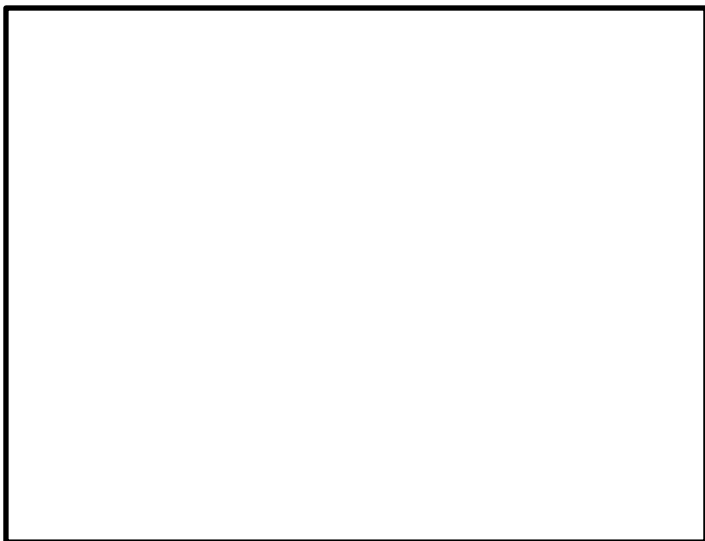
- A **type** defines a set of possible values.
 - e.g. **string**: "", "cat", "hello world!"
 - e.g. **boolean**: true, false
 - e.g. **Person**: {"james", 2, 110, 0xE5A4}
- **Static Typing**:
Variables have a fixed type.
- **Dynamic Typing**:
A variable may refer to different types.
The type is on the value only.
- The job of a type system:
 - What **operations** are available?
 - Which **conversions*** are allowed?
- **Strong Typing**
Few, if any, implicit conversions.
- **Weak Typing**
Eh, just do whatever.

*We usually think of this first in terms of **implicit conversions**, sometimes called coercions. Explicit casts may be used to circumvent the usual behavior of the type system (and sometimes its safety guarantees).

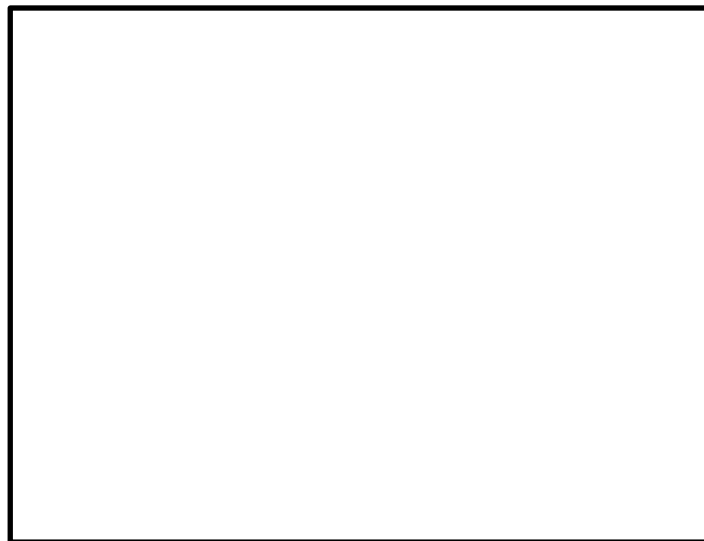


Value Semantics vs. Reference Semantics

C++



Typescript



Structural vs. Nominal Typing

Demo

Grading Policies

- **Lecture**

You may miss up to 4 lectures and still earn full credit.

- **Lab**

You may miss up to 2 labs and still earn full credit.

- **Project Checkpoints**

Ensure at least one member of your group is present at lab for checkpoints. Of course, the ideal is that everyone should attend.



Academic Integrity

- You're encouraged to collaborate.
- Don't submit others work as your own.
- Don't do others work for them.
- Earnestly engage with your project team.

AI Coding Tools

- AI usage is allowed, but you must document how and where it is used. This will help us give relevant feedback.
- We recommend against using agentic coding tools.
- Tools like Copilot that work more as an “autocomplete” are probably OK, but keep an eye on your own fluency.
- When using AI, use it to build your own knowledge and capabilities, not to replace yourself.



Exercise: Design Challenge

Exercise: Design Patterns



Break Time